

# SQL Avanzado

Joins, Agrupaciones, Subconsultas y Vistas

Motor: SQLite · Entorno: Google Colab · Docente: Pilar Rocío Sayán Mejía

## Contenido

|                                    |                                                 |
|------------------------------------|-------------------------------------------------|
| <b>01</b> Base de datos del Lab D3 | <b>02</b> JOIN — INNER, LEFT, RIGHT, FULL OUTER |
| <b>03</b> GROUP BY y HAVING        | <b>04</b> Subconsultas                          |
| <b>05</b> Vistas (VIEW)            | <b>06</b> Buenas prácticas                      |
| <b>07</b> Lab D3 completo          | <b>08</b> ¿Cuándo usar INNER JOIN vs LEFT JOIN? |

### 01 Base de datos del Lab D3: Clientes, Pedidos y Productos

El Lab D3 tiene tres tablas relacionadas que simulan el sistema de ventas de una empresa en Cusco.

#### Celda 1 — Conexión e importaciones

```
import sqlite3                # Módulo SQLite incluido en Colab
from tabulate import tabulate # Para mostrar tablas bonitas

conn = sqlite3.connect('labd3.db') # Crea o abre el archivo labd3.db
cur = conn.cursor()               # Cursor: el lápiz que ejecuta SQL

def ver(consulta):
    cur.execute(consulta)
    filas = cur.fetchall()
    cols = [d[0] for d in cur.description]
    print(tabulate(filas, headers=cols, tablefmt='grid'))
    print(f'{len(filas)} fila(s)\n')
```

#### Celda 2 — Crear las tres tablas

```

-- TABLA CLIENTES
CREATE TABLE IF NOT EXISTS clientes (
    id_cliente INTEGER PRIMARY KEY AUTOINCREMENT, -- PK único
    nombre TEXT NOT NULL, -- Nombre completo
    ciudad TEXT DEFAULT 'Cusco', -- Ciudad
    email TEXT UNIQUE -- Email único
);

-- TABLA PRODUCTOS
CREATE TABLE IF NOT EXISTS productos (
    id_producto INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre TEXT NOT NULL,
    categoria TEXT NOT NULL, -- Software, Hardware, Servicio...
    precio REAL NOT NULL -- Precio unitario en soles
);

-- TABLA PEDIDOS
CREATE TABLE IF NOT EXISTS pedidos (
    id_pedido INTEGER PRIMARY KEY AUTOINCREMENT,
    fecha TEXT NOT NULL, -- Formato: 'YYYY-MM-DD'
    cantidad INTEGER NOT NULL, -- Unidades compradas
    id_cliente INTEGER NOT NULL, -- FK → clientes
    id_producto INTEGER NOT NULL, -- FK → productos
    FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente),
    FOREIGN KEY (id_producto) REFERENCES productos(id_producto)
);
conn.commit() -- Guarda los CREATE TABLE

```

### Celda 3 — Insertar datos de ejemplo

```
-- Clientes (10 registros)
INSERT INTO clientes (nombre, ciudad, email) VALUES
('Maria Quispe',      'Cusco',      'mquispe@mail.com'), -- id=1
('Carlos Ttito',      'Cusco',      'cttito@mail.com'),  -- id=2
('Ana Mamani',        'Puno',        'amamani@mail.com'), -- id=3
('Pedro Huanca',      'Cusco',      'phuanca@mail.com'), -- id=4
('Lucia Condori',     'Arequipa',   'lcondori@mail.com'), -- id=5
... (10 clientes en total)

-- Productos (6 registros)
INSERT INTO productos (nombre, categoria, precio) VALUES
('Licencia Office 365', 'Software', 450.00),
('Laptop Dell i7',      'Hardware', 3200.00),
('Soporte Tecnico',     'Servicio', 250.00),
('Antivirus Anual',     'Software', 180.00),
('Disco Externo 2TB',   'Hardware', 320.00),
('Capacitacion SQL',    'Servicio', 500.00);

-- Pedidos (15 registros)
INSERT INTO pedidos (fecha, cantidad, id_cliente, id_producto) VALUES
('2024-01-10', 2, 1, 1), -- Maria compra 2 licencias Office
('2024-01-15', 1, 2, 2), -- Carlos compra 1 laptop
... (15 pedidos en total)
conn.commit()
print('Base de datos lista')
```

#### Diagrama de relaciones:

clientes (id\_cliente)  $\Leftarrow$  pedidos (id\_cliente · id\_producto)  $\Rightarrow$  productos (id\_producto)

Un cliente → MUCHOS pedidos | Un producto → MUCHOS pedidos

Monto = pedidos.cantidad × productos.precio

## 02 JOIN — Unir tablas: INNER, LEFT, RIGHT, FULL OUTER

| Tipo de JOIN    | ¿Qué devuelve?                                      | Filas sin match         |
|-----------------|-----------------------------------------------------|-------------------------|
| INNER JOIN      | Solo filas con coincidencia en AMBAS tablas         | Se descartan            |
| LEFT JOIN       | Todas de la izquierda + coincidencias de la derecha | NULL en col. derechas   |
| RIGHT JOIN      | Todas de la derecha + coincidencias izquierda       | NULL en col. izquierdas |
| FULL OUTER JOIN | Todas las filas de AMBAS tablas                     | NULL en ambos lados     |

### 2.1 INNER JOIN — Solo lo que coincide en ambas tablas

```

ver(''''
SELECT p.id_pedido          AS Pedido,
       p.fecha             AS Fecha,
       c.nombre            AS Cliente,  -- c = alias de clientes
       pr.nombre           AS Producto, -- pr = alias de productos
       p.cantidad          AS Qty,
       pr.precio           AS Precio_Unit,
       p.cantidad * pr.precio AS Monto_Total
FROM   pedidos p
INNER JOIN clientes c ON p.id_cliente = c.id_cliente
INNER JOIN productos pr ON p.id_producto = pr.id_producto
ORDER BY p.fecha;
''')

```

| Pedido | Fecha      | Cliente      | Producto            | Qty | Precio_Unit | Monto_Total    |
|--------|------------|--------------|---------------------|-----|-------------|----------------|
| 1      | 2024-01-10 | Maria Quispe | Licencia Office 365 | 2   | 450.0       | 900.0          |
| 2      | 2024-01-15 | Carlos Ttito | Laptop Dell i7      | 1   | 3200.0      | 3200.0         |
| 3      | 2024-01-20 | Maria Quispe | Antivirus Anual     | 3   | 180.0       | 540.0          |
| 4      | 2024-02-05 | Ana Mamani   | Soporte Tecnico     | 1   | 250.0       | 250.0          |
| ...    | ...        | ...          | ...                 | ... | ...         | 15 filas total |

## 2.2 LEFT JOIN — Todos los de la izquierda, aunque no tengan pedidos

```

ver(''''
SELECT c.nombre          AS Cliente,
       c.ciudad          AS Ciudad,
       COUNT(p.id_pedido) AS Total_Pedidos,
       COALESCE(SUM(p.cantidad*pr.precio), 0) AS Total_Comprado
FROM   clientes c
LEFT JOIN pedidos p ON c.id_cliente = p.id_cliente
LEFT JOIN productos pr ON p.id_producto = pr.id_producto
GROUP BY c.id_cliente, c.nombre, c.ciudad
ORDER BY Total_Comprado DESC;
''')

```

**Marco Salas aparece con Total\_Pedidos=0 y Total\_Comprado=0.**

Con INNER JOIN no aparecería. Con LEFT JOIN sí aparece.

Clave para reportes de clientes inactivos.

## 2.3 RIGHT JOIN — simulado con LEFT JOIN invertido (SQLite)

```
-- RIGHT JOIN en SQLite se simula invirtiendo el LEFT JOIN
ver(''')
SELECT c.nombre AS Cliente,
       p.fecha AS Fecha_Pedido,
       pr.nombre AS Producto
FROM   clientes c
LEFT JOIN pedidos p ON c.id_cliente = p.id_cliente
LEFT JOIN productos pr ON p.id_producto = pr.id_producto
ORDER BY c.nombre;
''')
```

## 2.4 FULL OUTER JOIN — simulado con UNION (SQLite)

```
ver(''')
SELECT c.nombre AS Cliente, p.id_pedido AS Pedido
FROM   clientes c
LEFT JOIN pedidos p ON c.id_cliente = p.id_cliente -- Todos los clientes
UNION
SELECT c.nombre AS Cliente, p.id_pedido AS Pedido
FROM   pedidos p
LEFT JOIN clientes c ON p.id_cliente = c.id_cliente -- Todos los pedidos
ORDER BY Cliente;
''')
```

### Resumen SQLite y JOINS:

INNER JOIN → soportado nativo    LEFT JOIN → soportado nativo  
 RIGHT JOIN → se simula con LEFT JOIN invertido  
 FULL OUTER JOIN → se simula con UNION de dos LEFT JOINS

## 03 GROUP BY y HAVING — Agrupar y filtrar grupos

**WHERE filtra filas ANTES de agrupar (actua sobre filas individuales).**

HAVING filtra grupos DESPUES de agrupar (actua sobre el resultado del GROUP BY).

Regla practica: si usas SUM, COUNT... en el filtro, usa HAVING.

### 3.1 Total de ventas por categoría de producto

```
ver(''')
SELECT pr.categoria AS Categoria,
       COUNT(p.id_pedido) AS Num_Pedidos,
       SUM(p.cantidad*pr.precio) AS Total_Ventas,
       ROUND(AVG(pr.precio), 2) AS Precio_Promedio
FROM   pedidos p
INNER JOIN productos pr ON p.id_producto = pr.id_producto
GROUP BY pr.categoria
ORDER BY Total_Ventas DESC;
''')
```

| Categoria | Num_Pedidos | Total_Ventas | Precio_Promedio |
|-----------|-------------|--------------|-----------------|
| Hardware  | 4           | 6720.0       | 1760.0          |
| Software  | 7           | 3870.0       | 315.0           |
| Servicio  | 4           | 2250.0       | 375.0           |

### 3.2 HAVING — Solo categorías con ventas > S/ 3 000

```
ver('')
SELECT pr.categoria           AS Categoria,
       SUM(p.cantidad * pr.precio) AS Total_Ventas
FROM   pedidos p
INNER JOIN productos pr ON p.id_producto = pr.id_producto
GROUP BY pr.categoria
HAVING SUM(p.cantidad * pr.precio) > 3000 -- HAVING filtra DESPUES de agrupar
ORDER BY Total_Ventas DESC;
''')
```

### 3.3 GROUP BY múltiple — Ventas por mes y ciudad

```
ver('')
SELECT SUBSTR(p.fecha, 1, 7)      AS Mes,
       c.ciudad                  AS Ciudad,
       COUNT(p.id_pedido)         AS Pedidos,
       SUM(p.cantidad * pr.precio) AS Total
FROM   pedidos p
INNER JOIN clientes  c  ON p.id_cliente = c.id_cliente
INNER JOIN productos pr ON p.id_producto = pr.id_producto
GROUP BY SUBSTR(p.fecha, 1, 7), c.ciudad
ORDER BY Mes, Total DESC;
''')
```

## 04 Subconsultas — Consultas dentro de consultas

Una subconsulta es un SELECT dentro de otro SELECT. Se coloca entre paréntesis y puede aparecer en el WHERE, FROM o SELECT.

### 4.1 Subconsulta en WHERE — Clientes que compraron más del promedio

```

ver('''
SELECT c.nombre                                AS Cliente,
      SUM(p.cantidad * pr.precio)             AS Total_Comprado
FROM   pedidos p
INNER JOIN clientes  c  ON p.id_cliente = c.id_cliente
INNER JOIN productos pr ON p.id_producto = pr.id_producto
GROUP BY c.id_cliente
HAVING SUM(p.cantidad * pr.precio) > (      -- Compara con subconsulta
      SELECT AVG(total_cliente) FROM (
        SELECT SUM(p2.cantidad * pr2.precio) AS total_cliente
        FROM   pedidos p2
        INNER JOIN productos pr2 ON p2.id_producto = pr2.id_producto
        GROUP BY p2.id_cliente
      )
    )
)
ORDER BY Total_Comprado DESC;
''')

```

## 4.2 Subconsulta en FROM — Clasificación de clientes con CASE

```

ver('''
SELECT resumen.Cliente,
      resumen.Total_Comprado,
      CASE
        WHEN resumen.Total_Comprado >= 5000 THEN 'VIP'
        WHEN resumen.Total_Comprado >= 2000 THEN 'Frecuente'
        ELSE 'Ocasional'
      END AS Categoria_Cliente
FROM (
  SELECT c.nombre                                AS Cliente,
        SUM(p.cantidad * pr.precio) AS Total_Comprado
  FROM   pedidos p
  INNER JOIN clientes  c  ON p.id_cliente = c.id_cliente
  INNER JOIN productos pr ON p.id_producto = pr.id_producto
  GROUP BY c.id_cliente
) AS resumen
ORDER BY Total_Comprado DESC;
''')

```

| Cliente      | Total_Comprado | Categoria_Cliente |
|--------------|----------------|-------------------|
| Maria Quispe | 5240.0         | VIP               |
| Carlos Ttito | 4000.0         | Frecuente         |
| Pedro Huanca | 1440.0         | Ocasional         |
| Ana Mamani   | 250.0          | Ocasional         |

## 4.3 Subconsulta en SELECT — Producto favorito por cliente

```

ver(''''
SELECT c.nombre AS Cliente,
      (SELECT pr.nombre
       FROM pedidos p2
       INNER JOIN productos pr ON p2.id_producto = pr.id_producto
       WHERE p2.id_cliente = c.id_cliente
       GROUP BY pr.id_producto
       ORDER BY SUM(p2.cantidad) DESC
       LIMIT 1) AS Producto_Favorito
FROM clientes c
WHERE c.id_cliente IN (SELECT DISTINCT id_cliente FROM pedidos);
''')

```

## 05 Vistas (VIEW) — Guardar y reutilizar consultas

**Una VIEW es una consulta SELECT guardada con un nombre en la base de datos.**

No almacena datos: cada vez que la consultas, ejecuta el SELECT original.

Se usa igual que una tabla: SELECT \* FROM nombre\_vista

### 5.1 Crear vista de ventas por cliente

```

cur.execute(''''
CREATE VIEW IF NOT EXISTS v_ventas_cliente AS
SELECT c.id_cliente,
      c.nombre                AS Cliente,
      c.ciudad                AS Ciudad,
      COUNT(p.id_pedido)      AS Total_Pedidos,
      SUM(p.cantidad * pr.precio) AS Total_Comprado,
      MAX(p.fecha)            AS Ultima_Compra
FROM   clientes c
LEFT JOIN pedidos  p ON c.id_cliente = p.id_cliente
LEFT JOIN productos pr ON p.id_producto = pr.id_producto
GROUP BY c.id_cliente, c.nombre, c.ciudad
''')
conn.commit()
print('Vista creada')

```

### 5.2 Usar la vista — igual que una tabla



```

ver('SELECT * FROM v_ventas_cliente ORDER BY Total_Comprado DESC')

ver(''
SELECT Cliente, Ciudad, Total_Comprado
FROM   v_ventas_cliente
WHERE  Ciudad = 'Cusco'
AND    Total_Pedidos > 0
ORDER BY Total_Comprado DESC
LIMIT 5;
'')

```

### 5.3 Crear vista de ranking de productos

```

cur.execute(''
CREATE VIEW IF NOT EXISTS v_productos_ranking AS
SELECT pr.id_producto,
       pr.nombre           AS Producto,
       pr.categoria        AS Categoria,
       pr.precio           AS Precio_Unit,
       COUNT(p.id_pedido)  AS Veces_Vendido,
       SUM(p.cantidad)     AS Unidades_Total,
       SUM(p.cantidad * pr.precio) AS Ingresos_Total
FROM   productos pr
LEFT JOIN pedidos p ON pr.id_producto = p.id_producto
GROUP BY pr.id_producto
'')
conn.commit()
ver('SELECT * FROM v_productos_ranking')

```

### 5.4 Eliminar una vista

```

cur.execute('DROP VIEW IF EXISTS v_ventas_cliente') # Elimina la vista
conn.commit()

```

## 06 Buenas prácticas: alias, legibilidad y comentarios

### 6.1 Alias: nombres cortos y descriptivos

```
-- MAL: sin alias, difícil de leer
SELECT clientes.nombre, pedidos.fecha, productos.precio
FROM clientes INNER JOIN pedidos
  ON clientes.id_cliente = pedidos.id_cliente
INNER JOIN productos ON pedidos.id_producto = productos.id_producto;

-- BIEN: con alias claros y columnas renombradas
SELECT c.nombre AS Cliente,
       p.fecha   AS Fecha_Compra,
       pr.precio AS Precio_Soles
FROM   clientes c
INNER JOIN pedidos  p ON c.id_cliente = p.id_cliente
INNER JOIN productos pr ON p.id_producto = pr.id_producto;
```

## 6.2 Buenas prácticas de formato

| Práctica                    | Evitar                                    | Preferir                          |
|-----------------------------|-------------------------------------------|-----------------------------------|
| Palabras clave              | select * from pedidos                     | SELECT * FROM pedidos             |
| Una cláusula por línea      | SELECT a FROM b WHERE c<br>GROUP BY d     | SELECT a\nFROM b\nWHERE c         |
| Nombrar columnas calculadas | SUM(cantidad * precio)                    | SUM(cantidad*precio) AS Total     |
| Alias de tablas             | INNER JOIN clientes ON<br>clientes.id=... | INNER JOIN clientes c ON c.id=... |

## 07 Lab D3 completo — Análisis de ventas con JOINS

**Objetivo: Aplicar todos los conceptos del manual sobre labd3.db.**

Responder 5 preguntas de negocio usando JOINS, GROUP BY, HAVING, subconsultas y vistas.

### Consulta D3-1: TOP 5 clientes por monto total comprado

```
ver(''''
/* LAB D3 – Consulta 1: Top 5 clientes */
SELECT c.nombre                AS Cliente,
       c.ciudad                AS Ciudad,
       COUNT(p.id_pedido)      AS Num_Pedidos,
       SUM(p.cantidad)         AS Unidades_Total,
       SUM(p.cantidad * pr.precio) AS Monto_Total,
       MAX(p.fecha)            AS Ultima_Compra
FROM   pedidos p
INNER JOIN clientes  c ON p.id_cliente = c.id_cliente
INNER JOIN productos pr ON p.id_producto = pr.id_producto
GROUP BY c.id_cliente, c.nombre, c.ciudad
ORDER BY Monto_Total DESC
LIMIT 5;
''')
```

| Cliente      | Ciudad | Num_Pedidos | Unidades_Total | Monto_Total | Ultima_Compra |
|--------------|--------|-------------|----------------|-------------|---------------|
| Maria Quispe | Cusco  | 4           | 7              | 5240.0      | 2024-05-20    |
| Carlos Ttito | Cusco  | 3           | 3              | 4000.0      | 2024-06-01    |
| Pedro Huanca | Cusco  | 2           | 5              | 1440.0      | 2024-05-01    |
| Jose Apaza   | Cusco  | 1           | 4              | 720.0       | 2024-03-10    |
| Sofia Ccopa  | Lima   | 1           | 2              | 500.0       | 2024-04-05    |

### Consulta D3-2: Productos más vendidos por categoría

```
ver('''
SELECT pr.categoria          AS Categoria,
       pr.nombre             AS Producto,
       SUM(p.cantidad)        AS Unidades_Vendidas,
       SUM(p.cantidad*pr.precio) AS Ingresos
FROM   pedidos p
INNER JOIN productos pr ON p.id_producto = pr.id_producto
GROUP BY pr.categoria, pr.id_producto
ORDER BY pr.categoria, Ingresos DESC;
''')
```

### Consulta D3-3: Clientes sin pedidos (LEFT JOIN + IS NULL)

```
ver('''
SELECT c.nombre AS Cliente_Inactivo,
       c.ciudad AS Ciudad,
       c.email AS Email
FROM   clientes c
LEFT JOIN pedidos p ON c.id_cliente = p.id_cliente
WHERE  p.id_pedido IS NULL;  -- NULL = sin ningun pedido
''')
```

| Cliente_Inactivo | Ciudad | Email |
|------------------|--------|-------|
| Marco Salas      | Cusco  | NULL  |

### Consulta D3-4: Análisis mensual de ventas

```

ver('')
SELECT SUBSTR(p.fecha, 1, 7)           AS Mes,
       COUNT(DISTINCT p.id_cliente)   AS Clientes_Activos,
       COUNT(p.id_pedido)             AS Num_Pedidos,
       SUM(p.cantidad * pr.precio)     AS Ventas_Mes,
       ROUND(AVG(p.cantidad * pr.precio), 2) AS Ticket_Promedio
FROM   pedidos p
INNER JOIN productos pr ON p.id_producto = pr.id_producto
GROUP BY SUBSTR(p.fecha, 1, 7)
ORDER BY Mes;
''')

```

### Consulta D3-5: Reporte final con segmentación

```

ver('')
SELECT Cliente, Ciudad, Total_Pedidos, Total_Comprado, Ultima_Compra,
       CASE
           WHEN Total_Comprado >= 5000 THEN 'VIP'
           WHEN Total_Comprado >= 2000 THEN 'Frecuente'
           WHEN Total_Comprado > 0      THEN 'Ocasional'
           ELSE                          'Sin compras'
       END AS Segmento
FROM   v_ventas_cliente
ORDER BY Total_Comprado DESC;
''')

```

## 08 ¿Cuándo usar INNER JOIN vs LEFT JOIN? — Resumen final

| Pregunta de negocio                        | JOIN recomendado    | ¿Por qué?                        |
|--------------------------------------------|---------------------|----------------------------------|
| ¿Qué clientes han comprado?                | INNER JOIN          | Solo los que tienen pedidos      |
| ¿Todos los clientes, hayan comprado o no?  | LEFT JOIN           | Incluye clientes sin pedidos     |
| ¿Qué productos se han vendido?             | INNER JOIN          | Solo los que aparecen en pedidos |
| ¿Todos los productos, incluso no vendidos? | LEFT JOIN           | Muestra productos sin ventas     |
| Reporte de ventas del mes                  | INNER JOIN          | Solo transacciones reales        |
| Buscar registros huérfanos                 | LEFT JOIN + IS NULL | Detecta datos sin coincidencia   |
| Comparar listas de dos tablas              | FULL OUTER JOIN     | Ver diferencias entre tablas     |

### ¿Por qué en D3-1 usamos INNER JOIN y no LEFT JOIN?

→ Porque solo queremos clientes QUE SI compraron. Un cliente sin pedidos no tiene monto.

### ¿Si quisiéramos ver también clientes que nunca compraron?

→ Usaríamos LEFT JOIN y COALESCE(SUM(...), 0) para que aparezcan con S/ 0.00

| Comando       | ¿Para qué sirve?                            | Ejemplo                       |
|---------------|---------------------------------------------|-------------------------------|
| INNER JOIN    | Solo filas con coincidencia en ambas tablas | JOIN pedidos p ON c.id=p.id_c |
| LEFT JOIN     | Todas las de la izquierda + coincidencias   | LEFT JOIN pedidos p ON ...    |
| UNION         | Combina resultados de dos SELECT            | SELECT a UNION SELECT b       |
| GROUP BY      | Agrupar filas para funciones de resumen     | GROUP BY ciudad               |
| HAVING        | Filtrar grupos (después de GROUP BY)        | HAVING SUM(monto) > 1000      |
| CASE WHEN     | Condición dentro de SELECT                  | CASE WHEN x>5 THEN 'A'        |
| CREATE VIEW   | Guarda consulta con nombre reutilizable     | CREATE VIEW v AS SELECT ...   |
| COALESCE(x,0) | Reemplaza NULL por valor por defecto        | COALESCE(SUM(m), 0)           |
| SUBSTR(f,1,7) | Extrae parte de un texto (año-mes)          | SUBSTR('2024-05-10',1,7)      |
| ROUND(v, n)   | Redondea a n decimales                      | ROUND(AVG(precio), 2)         |